SAT Rapidity Frontier: Current Formal State and Remaining Bridge

HQIV Lean notes

May 1, 2026

Purpose

This note summarizes the current formal state of the SAT rapidity program in the repository, from the bottom-level certified facts already in Lean up to the final missing geometric bridge. The goal is to phrase the setup clearly enough to think through the last step.

1 Bottom layer: what is already certified

The SAT rapidity stack is organized as *several* layers (worst-case semantics, shared manifold / rapidity, residual control, and later—packing, direction selection, exact-count/plane bridge, ribbon-cover collapse). The first three are summarized here.

1.1 SAT worst-case certification layer

In `Hqiv/Geometry/SATWorstCaseCertified.lean`, the repository already has:

- sound-prune semantics: pruning never removes a satisfying assignment;
- survivor-exhaustion semantics: exhaustive evaluation of the surviving pool with no model implies UNSAT;
- work-envelope transfer: if survivor work is bounded by a baseline plus an additive residual budget ε, and ε is within the usual root-scale budget, then the normalized survivor work lies inside the same $1 + n^{1/n}$ envelope used on the ATSP side;
- cumulative arity-residual transfer:

$$\sum_i \varepsilon_i \le \text{baselineWork} \cdot n^{1/n} \quad \Longrightarrow \quad \frac{\text{survivorWork}}{\text{baselineWork}} \le 1 + n^{1/n};$$

- a polynomial-budget hook: if both baseline work and cumulative residual budget are polynomially bounded, then survivor work is polynomially bounded.

So the SAT file already gives the *formal landing zone* for a polynomial-time-style argument. What it does *not* supply is the geometric theorem that forces those polynomial bounds.

1.2 Shared manifold / rapidity layer

In `Hqiv/Geometry/SATRapidityManifold.lean`, we now formalize the idea that:

- variables and clauses both embed into a shared ambient manifold;
- there is one common scalar rapidity observable on that manifold;
- the variable-side and clause-side rapidity profiles are both read from that same observable;
- a common rapidity threshold can therefore control both sides at once.

At the current scaffold level, rapidity is phrased in the simplest successor-step form: a discrete $m \mapsto m + 1$ combinatorial step read from the shared manifold.

Concretely, the file now contains:

- a shared manifold structure `SATSharedManifold`;
- rapidity profiles on the variable and clause sides;
- successor-step rapidity channels `variableRapidityStep` and `clauseRapidityStep`;
- common-threshold lemmas in both profile and successor-step form;
- successor-style envelope constants and transfer lemmas for varDim + 1 and clauseDim + 1, mirroring the ATSP $n + 1$ route.

1.3 Residual-control layer

The same file also introduces the first serious theorem-start for the missing bridge:

> **Successor-step residual control:** each SAT residual is indexed by either a variable-side or clause-side successor step, and that residual is bounded by the corresponding rapidity step, which is itself bounded by one common threshold.

Formally this is the structure `SuccessorStepResidualControl`. From it, the repository already proves:

1. every residual is bounded by the common threshold;
2. therefore one can build a `SATSharedRapidityCertificate`;
3. if the threshold is polynomially bounded and the residual list length is polynomially bounded, then the cumulative residual budget is polynomially bounded.

Thus rapidity can *state* successor-step control cleanly; what a complete SAT instance theorem still needs is a *geometric or encoding witness* linking that control to a bounded frontier (the combinatorial implications of such a witness are largely already packaged higher in the stack—see §4).

2 Why stars-and-bars is probably not enough

If the frontier bound is treated as a generic occupancy count, one gets something like stars-and-bars behavior. That is too weak for the intended result: it does not force a sharp collapse of the frontier, and it does not naturally explain why a common rapidity threshold should drastically reduce the candidate family.

So the current working view is:

> The missing theorem is probably not a bare combinatorial counting lemma. It is more likely a packing theorem.

In other words, the residual directions should be thought of as a family of directions, rays, tangent vectors, or shell points on a common manifold, and the threshold should force them to be sufficiently separated that only finitely many can coexist.

3 Frontier geometry: the new packing layer

3.1 Analytical arc, annular ribbon, and moiré

On the analytical sphere, the relevant continuum between a pole and a k-arity pole is still a **one-dimensional arc** γ. In Lean, the current counting kernel is expressed through a local 2-dimensional section (`Hqiv/Geometry/SATRapidityAnnulusCircle.lean`); this should be read as an *osculating ribbon section*, not as a global claim that the ambient manifold itself is literally planar or globally spherical. In that local section, the arc lies on the target shell circle C_M. Discrete candidates lie in a tubular **ribbon** (points within distance ε of some $\gamma(t)$); **moiré** here means that a lattice point in the ribbon generically does *not* lie on γ itself. The radial annulus $A_M(\tau)$ captures these near-arc points once $\varepsilon \leq \tau$; this inclusion is proved in Lean from the triangle inequality (`inAnnulus_of_shell_arc_ribbon`).

The current bridge file `Hqiv/Geometry/SATRapidityPlaneBridge.lean` makes this local nature explicit: `PlaneWitnessMap` is the data `ResidualPoint M → Plane`, and the same object is also named `LocalRibbonSectionWitness` (definitionally the same structure, with identity coercions) to stress that the global manifold remains abstract while the counting kernel lives in a chosen osculating 2-plane / local ribbon section.

In `Hqiv/Geometry/SATRapidityPackingBridge.lean`, the next frontier has now been formalized.

The new object is a packing-style certificate:

```
SATRapidityPackingCertificate
```

with the intended interpretation:

- the residual frontier lives in an effective dimension d;
- there is a packing / kissing-number style bound frontierBound(d);
- the residual list length is bounded by that frontier bound;
- that bound is controlled by the combined variable/clause dimension.

The main theorem already proved from that scaffold is:

packing control + successor-step residual control $\Longrightarrow$ polynomial residual budget.

So, structurally, the chain is now:

shared manifold $\Longrightarrow$ successor-step rapidity $\Longrightarrow$ residual domination $\Longrightarrow$ packing-controlled frontier length $\Longrightarrow$ po

The same file stack also supports a *parallel* route through planar exact-count bounds ($K_{\text{exact}} \le 2|Q|$) and `RibbonCoverCollapseData`; that route does not require an abstract kissing-number function $K(d)$ once the local shell hypotheses hold.

4 What is now fully constructed in Lean

Since the original version of this note, the intermediate bridge has become substantially more explicit.

4.1 Direction selection, gap bridge, and geometric collapse are packaged

The repository now contains:

- `DirectionSelectionCertificate`: residuals choose canonical shell directions, with injectivity/separation packaged as explicit fields;
- `SATRapidityGapBridge`: the single record representing the statement "rapidity gap controls canonical directions";
- `SATRapidityGeometricCollapse`: the end-stage record packaging logarithmic effective dimension, sphere-code control, and successor-step bookkeeping.

Moreover, once the appropriate hypotheses are supplied (including via the plane-cover and ribbon-cover constructors in `SATRapidityPlaneBridge.lean`), one can build all three levels by explicit constructors rather than only by isolated `theorem` calls.

4.2 Exact-count route is now explicit

The exact-count route through local shell intersections has been formalized in `SATRapidityExactCardinality.lea` and wired to the planar model in `SATRapidityAnnulusCircle.lean` / `SATRapidityPlaneBridge.lean`.

The key proved ingredients are:

1. for each nonzero shell center q, a local shell fiber satisfying the `planeLocalShellIntersections` predicate obeys $|I| \le 2$ (circle–circle geometry);
2. therefore the exact admissible-direction count
$$K_{\text{exact}} := \#\Big(\bigcup_{q \in Q} \text{intersections}(q)\Big)$$
satisfies $K_{\text{exact}} \le 2|Q|$;
3. if residual list length is dominated by K_{exact}, then it is bounded by $2|Q|$, and hence by $2B$ whenever $|Q| \le B$;

4. when Q is the carrier of an `ArcRibbonLatticeCardBound` aligned with the annulus model, the same chain yields len $\leq 2 \cdot$ countBound$(\cdot)$ without choosing an intermediate B by hand;

5. natural-number and real-cast versions feed the `hSphereLen` slot once combined with a frontier comparison into `sphereCodeBound effectiveDim`.

The bridge file contains *several* explicit constructor families, all hypothesis-explicit:

- **Comparison frontier:** `DirectionSelectionCertificate.of_plane_cover_bound`, `SATRapidityGapBridge` `SATRapidityGeometricCollapse.of_plane_cover_bound`, assuming $2B \leq$ sphereCodeBound(effectiveDim) (as reals).
- **Exact frontier:** the parallel `of_plane_cover_exact_frontier` constructors when sphereCodeBound(effective $2B$, so no separate inequality step is needed at that dimension.
- **Threading only:** `direction_selection_with_plane_witness_implies_gap_bridge` and `direction_select` repackage an *existing* certificate and only carry a `PlaneWitnessMap` for joint hypotheses (they do not derive `hSphereLen` from the plane).

A small finite-size lemma internal to the same file records that the factorization/oracle `candidateList` has length exactly $3($`candidateStepBound`$(n) + 1)$ (`factorization_candidate_family_card_eq`), packaging the list-length part of that pipeline into the SAT rapidity stack (membership in an annulus family is still separate).

4.3 Single-hypothesis collapse of steps 2–5

The strongest consolidated packaging is the structure

```
RibbonCoverCollapseData M D control
```

in `SATRapidityPlaneBridge.lean`. It assumes an ambient `DirectionSelectionCertificate D` and `SuccessorStepResidualControl control`, and bundles:

- `witness` : `LocalRibbonSectionWitness M` (same as `PlaneWitnessMap`);
- `family` : `AnnulusLatticeFamily` at (`shellRadius`, `thresholdWidth`) from `D.annulusModel`;
- `intersections` : `Plane → Finset Plane` and the fiber hypotheses `hne`, `hloc` (`planeLocalShellIntersectio` on each center);
- cover `hCover` : residual centers land in `family.carrier`;
- exact-count domination `hK` : `D.residuals.length` $\leq K_{\text{exact}}$;
- polynomial slot `countBound`, `varDim`, `clauseDim`, `tauBits`, and `hCard` : |`family.carrier`| $\leq$ `countBound`$(\cdots)$;
- real comparison `hCountFrontier` : $2\cdot$`countBound`$(\cdots) \leq$ `D.sphereCodeBound(effectiveDim)`;
- successor packaging `hShared`, `hLen`, and `hPoly` for the sphere-code step.

From this record, Lean defines:

- `RibbonCoverCollapseData.hSphereLen` (theorem): discharges the sphere-length inequality for `D`;
- `toDirectionSelectionCertificate`: `D` with `hSphereLen` replaced by that proof;
- `toGapBridge`, `toGeometricCollapse`;
- named entry points `ribbon_cover_collapse`, `ribbon_cover_collapse_to_gap_bridge`, `ribbon_cover_collap` `ribbon_cover_collapse_to_sphere_code`;
- `ribbon_cover_collapse_hasPolynomialResidualBudget`: closes into `HasPolynomialResidualBudget` under the usual nonnegativity / threshold hypotheses;
- `ribbon_cover_collapse_implies_nat_root_envelope`: combines the collapsed residual budget with the certified ATSP-style envelope (`ATSPWorstCaseCertified`) for a single numeric worst-case bound.

The method `toGeometricCollapse` applies `DirectionSelectionCertificate.toGeometricCollapse` to the ambient `D`; the method `toDirectionSelectionCertificate` replaces `D.hSphereLen` with the theorem `RibbonCoverCollapseData.hSphereLen`. In applications where the ribbon witness is the authoritative justification for the sphere-length bound, the upgraded certificate should be used when passing into collapse or sphere-code packaging.

So the downstream pipeline after `RibbonCoverCollapseData` is fully wired in Lean; the remaining substantive task is still to *construct* that witness (and the scaffold interfaces below) from first-principles geometry and encoding.

5 What the final missing piece must say

At this point, the *formal* pipeline after a suitable witness is largely implemented in Lean (§4). What remains unproved is sharp in a different sense: one must *construct* the witness from SAT geometry and encoding, not reprove the downstream certificate algebra.

One seeks a theorem of the following shape.

Desired frontier theorem

There exists a natural effective dimension d attached to the shared variable/clause manifold such that the residual directions generated by the SAT oracle admit a local ribbon-section cover, and hence form a packing/counting family on a shell, tangent sphere, or comparable local geometric carrier. The number of admissible residual directions is then bounded by a function $K(d)$ of packing/kissing-number or exact-shell-count type.

If additionally $K(d)$ is polynomially controlled by the combined input dimension varDim + clauseDim, then the entire residual frontier is polynomially bounded.

Why this is the right target

This would supply what the formal pipeline is already set up to consume:

1. identify each residual with a local ribbon-section center and associated shell data;
2. show thresholded rapidity forces the hypotheses used in the planar/exact-count layer (nontrivial centers, `planeLocalShellIntersections` fibers, cover in a finite annulus family);

3. establish the exact-count domination len $\leq K_{\text{exact}}$ and a polynomial cardinality bound on $\#Q$ (e.g. via `ArcRibbonLatticeCardBound`);
4. package these as a `RibbonCoverCollapseData` witness (or equivalent inputs to `of_plane_cover_bound` / `of_plane_cover_exact_frontier`);
5. then *already formalized*: `RibbonCoverCollapseData` → `SATRapidityGeometricCollapse` → `SATRapiditySphereCodeCertificate` → `HasPolynomialResidualBudget` (see `ribbon_cover_collapse_hasP` and hence the certified SAT work / polynomial-budget story at the worst-case layer.

A separate, older abstraction `SATRapidityPackingCertificate` in `SATRapidityPackingBridge.lean` still represents a generic packing/kissing-style frontier bound; the ribbon-cover route in `SATRapidityPlaneBridge.l` closes through the sphere-code certificate instead. Either style of frontier input can feed polynomial-budget machinery once the numeric hypotheses match the downstream lemmas.

Equivalently: the *implications* after a ribbon-cover witness are already proved; what is still missing is the *theorem that produces* that witness from first principles.

6 Interpretive note on dimension and collapse

The motivating speculation is that the bridge may depend on a nontrivial dimension regime, not monotone growth with dimension. In that picture, one does *not* want a naive "more dimensions means more space" argument. Rather, one wants a regime in which the shared manifold geometry imposes a sharp structural bound on admissible frontier directions.

That is why sphere packing / kissing-number heuristics are more promising here than generic stars-and-bars counting. The right theorem would exploit geometry strong enough to turn the frontier into a constrained packing problem instead of an unconstrained occupancy problem.

7 Two concrete candidate approaches for the final bridge

The current best two candidate approaches are the following.

7.1 Angular separation on the rapidity sphere / local section

This is the most direct geometric route.

1. Map each residual to a unit vector or local shell direction whose direction is the gradient of the shared rapidity observable (or its local ribbon-section representative).
2. Use the common threshold to force angular separation between distinct residual directions. A model inequality would be
$$|\cos\theta_{ij}| \leq 1 - \delta$$
for distinct residuals $i \neq j$.
3. View the residual family as a spherical code on the unit sphere in $\mathbb{R}^d$, where d is derived from varDim + clauseDim.
4. Apply a spherical-code or kissing-number bound $K(d)$ to conclude that the frontier cardinality is at most $K(d)$.

This is attractive because it connects almost immediately to the existing packing certificate abstraction: $K(d)$ can serve as the `frontierBound` in `SATRapidityPackingCertificate`.

7.2 Riemannian tangent-space packing

This is the more general manifold-native route.

1. Equip the shared manifold with a Riemannian metric structure compatible with the existing oracle language.
2. Interpret residuals as tangent vectors with norm bounded by the successor-step rapidity.
3. Use the threshold to enforce a minimum angle between any two such tangent vectors.
4. Apply a tangent-space packing lemma: for example, one has exponential-type coarse bounds such as 2^d under large pairwise angular separation, and then one tries to improve these to polynomial-in-d bounds under additional curvature, injectivity-radius, or shell-regularity assumptions.

This route is more flexible, because it can absorb curvature and local manifold data, but the final theorem may need stronger geometric hypotheses than the spherical-code route.

8 What is still genuinely missing

Despite the progress above, a fully internal proof still lacks the following substantive ingredient.

8.1 The one remaining witness-producing theorem

The current Lean development still does *not* derive from first principles that:

1. every residual lies in a concrete local ribbon-section annulus family,
2. the associated shell fibers are the right admissible local intersections,
3. residual length is dominated by the resulting exact-count union model, and
4. the resulting annulus family carries the required polynomial cardinality witness.

This whole cluster is isolated as the construction of one `RibbonCoverCollapseData` witness (or, equivalently, the inputs to the `of_plane_cover_bound` constructor family together with the separate hypotheses that define a ribbon-cover collapse in the formal sense).

8.2 What is still scaffold-level

Two definitions remain intentionally lightweight placeholders in the abstract SAT scaffold:

- `localShellIntersections` on the fully abstract manifold side is still the empty-finset placeholder;
- `angle` is still the constant-0 placeholder on abstract shell directions.

These do *not* break the current bridge construction, because the new exact-count work is done in the local ribbon-section model. But a fully geometric final theorem should eventually replace those placeholder interfaces by real shell-intersection and angle data, or else prove that the local ribbon-section construction is the only geometry actually needed.

How they fit the current Lean scaffold

A full internal proof still has to supply a frontier bound and the encoding-side hypotheses above. Once those are available, the **primary wired endpoint** in the current development is:

- build a `RibbonCoverCollapseData` witness (or the equivalent plane-cover inputs);
- obtain `SATRapidityGeometricCollapse` and `SATRapiditySphereCodeCertificate`;
- apply `ribbon_cover_collapse_hasPolynomialResidualBudget` to reach `HasPolynomialResidualBudget`.

Separately, a classical packing-style certificate `SATRapidityPackingCertificate` can still serve as an abstract `frontierBound` route into polynomial-budget lemmas when its numeric hypotheses are arranged to match `packingCertificate_hasPolynomialResidualBudget`.

So the open choice is less about "which Lean record to use at the end" than about *which geometric theorem produces the frontier and cover data.* A spherical-code bound, a tangent-space packing bound, or the proved planar exact-count $\leq 2\#Q$ chain are different *sources* for those hypotheses; the repository already implements the bookkeeping once the bounds are supplied.

9 Best anchor points for the next proof attempt

After reviewing the agent-facing documentation, the best anchor points for the remaining work are now fairly clear.

9.1 Primary anchor: manifold rapidity roadmaps

The strongest direct anchor is the manifold roadmap:

- `AGENTS/MANIFOLD_ZETA_ROADMAP.md`

This document already states, in essentially the right form, that the missing theorem is a *ray theorem*: rapidity should factor through or agree with a canonical map from shell index to a unit tangent direction on a Riemannian slice. That is almost exactly the geometric-collapse problem now represented by `SATRapidityGeometricCollapse`.

So the most natural theorem target is not a totally new idea, but a SAT-specialized version of the same roadmap question:

> Can the shared SAT rapidity observable be shown to factor through a low-dimensional family of canonical directions (rays / tangent vectors / shell points) on the manifold, with enough angular separation to force a packing bound?

9.2 Secondary anchor: theorem index for existing reusable pieces

The next best anchor is:

- `AGENTS/THEOREMS.md`

The most relevant existing names are:

- Euclidean and shell-volume anchors from `SpatialSliceManifold`;

- the rapidity / polar-angle identification from `RapidityZetaPhaseBridge`;
- the explicit shell-to-$m{+}1$ monotonicity and shell-surface proxies from `OctonionSphereConstruction` and related modules;
- the Fourier patch / moiré slope language listed in the archive and modular-curvature notes.

These suggest that the next proof should be phrased less as "find a new object" and more as "bind together already-existing shell, phase, and patch concentration channels into one direction-selection theorem."

9.3 Third anchor: modular / Fourier patch narrative

The file

- `AGENTS/MODULAR_THETA_CURVATURE_BRIDGE.md`

is especially relevant because it explicitly points to Fourier patch concentration and curved-manifold lifts via heat kernels / Laplacian spectrum. This is a strong hint that, if the SAT rapidity frontier really collapses, a plausible mechanism may be:

1. moiré / patch concentration isolates a low-dimensional family of dominant directions,
2. rapidity phase selects among those directions,
3. packing then bounds how many such dominant directions can coexist.

9.4 Fourth anchor: Euler/Gauss-style shell intersection counts

There is also a simpler and potentially very powerful anchor: lattice intersection counts per shell.

The existing corpus does not yet contain a full Gauss circle-problem theorem in dimension two, but it does contain the right style of ingredients:

- discrete shell counts such as `r8` in `Hqiv/Algebra/IntegerLatticeShellCount8.lean`;
- shell monotonicity and continuous sphere/ball proxies in `OctonionSphereConstruction`;
- Euclidean shell / slice geometry in `SpatialSliceManifold` and `EuclideanBallHorizontalSlice`;
- the explicit warning in the docs that current combinatorics are incremental shell counts, not yet full Gauss-circle totals.

This suggests another framing of the direction-selection theorem:

> Instead of proving the frontier bound directly from abstract packing, prove that the annulus-shell construction reduces the residual frontier to a controlled lattice intersection problem on one shell, and then bound the number of admissible intersections by an Euler/Gauss-style shell-count function.

In the local 2D picture, each residual point in the annulus probes the target shell by a unit circle, and the canonical directions are the shell intersection points. The natural counting question is then:

How many such intersection points can occur on a fixed shell under the threshold and separation rules?

That is extremely close in spirit to a circle-problem or shell-count question, and may give a simpler route than a fully general manifold packing theorem if the relevant shell counts can be controlled sharply.

Recommended next Lean move

The next substantive Lean goal is unchanged in *name*—produce a `RibbonCoverCollapseData` witness (or equivalent `of_plane_cover_*` inputs) from shared-manifold, rapidity, and encoding hypotheses—but the *downstream* implications are already implemented (§4), so new code should focus on geometry and `hK`, not on re-proving certificate projections.

If the Euler/Gauss-shell route is pursued, a companion module such as `Hqiv/Geometry/SATRapidityShellInte` could package shell-intersection counting as an alternate source of frontier bounds feeding the same constructors.

10 Current status in one sentence

The repository has the full formal scaffold from shared manifold rapidity through direction selection, gap bridge, and geometric collapse; the exact-count / planar-fiber inequalities and the `RibbonCoverCollapseData` packaging (including closure into `HasPolynomialResidualBudget` and the ATSP-style root envelope) are implemented in Lean. The remaining substantive work is to construct the ribbon-cover / encoding witness and to replace abstract-placeholder `localShellIntersections` and `angle` with geometric content where a global statement is desired.

Files to consult

- `Hqiv/Geometry/SATWorstCaseCertified.lean`
- `Hqiv/Geometry/SATRapidityManifold.lean`
- `Hqiv/Geometry/SATRapidityDirectionSelection.lean`
- `Hqiv/Geometry/SATRapidityExactCardinality.lean`
- `Hqiv/Geometry/SATRapidityAnnulusCircle.lean`
- `Hqiv/Geometry/SATRapidityPackingBridge.lean`
- `Hqiv/Geometry/SATRapiditySpherePacking.lean`
- `Hqiv/Geometry/SATRapidityTangentPacking.lean`
- `Hqiv/Geometry/SATRapidityGapBridge.lean`
- `Hqiv/Geometry/SATRapidityPlaneBridge.lean` (ribbon-cover collapse, constructors, polynomial closure)
- `Hqiv/Geometry/ATSPWorstCaseCertified.lean` (envelope used by `ribbon_cover_collapse_implies_nat_ro`
- `Hqiv/Geometry/SpatialSliceRapidityScaffold.lean`

- `Hqiv/Geometry/GeneralRiemannianRapidityOracle.lean`